



INSTITUTO POLITÉCNICO DA UNIVERSIDADE KIMPA VITA

Licenciatura em Engenharia Informática

Unidade Curricular: **Programação Orientada a Objectos II**

PROPOSTA ACADÉMICA DE AVALIAÇÃO PRÁTICA

Desenvolvimento de Projectos em C# *Console Application*

3.º Ano – 2.º Semestre

Joaquim João Nsaku Ventura, MSc.

2025/2026

1. Título Geral da Avaliação Prática.....	3
2. Enquadramento Académico e Justificativo Pedagógico	3
3. Objectivo Geral da Avaliação.....	3
4. Objectivos Específicos.....	4
5. Competências Técnicas e Científicas a Avaliar	4
5.1. Competências de Análise e Modelação.....	4
5.2. Competências de Implementação	4
5.3. Competências de Qualidade de Software	5
5.4. Competências Comunicacionais e Documentais.....	5
6. Regras de Organização dos Grupos.....	5
7. Condições Técnicas Obrigatórias	5
8. Lista de 34 Projectos	6
9. Critérios Obrigatórios de Implementação.....	36
9.1. Encapsulamento	36
9.2. Herança	36
9.3. Polimorfismo	36
9.4. Classes Abstractas e Interfaces	36
9.5. Composição e Agregação	36
9.6. Colecções Genéricas.....	36
9.7. Tratamento de Excepções.....	37
9.8. Validação de Dados.....	37
9.9. Separação de Responsabilidades	37
9.10. Organização em Pastas e Namespaces.....	37
10. Estrutura Recomendada para Cada Projecto em C#.....	37
11. Orientações para a Elaboração do Diagrama de Classes UML.....	38
11.1. Representação de Classes.....	38
11.2. Atributos e Métodos.....	38

11.3. Relações entre Classes	38
11.4. Multiplicidades	39
11.5. Interfaces	39
12. Requisitos de Documentação	39
12.1. Estrutura do Relatório Técnico	39
13. Critérios de Avaliação com Pesos Percentuais.....	40
14. Modelo de Grelha de Avaliação (0–20 Valores).....	41
15. Orientações para a Apresentação Pública.....	42
15.1. Duração e Estrutura.....	42
15.2. Demonstração do Sistema	42
15.3. Explicação do Modelo UML	42
15.4. Demonstração do Código	42
15.5. Defesa Técnica.....	42
16. Resultados Esperados da Avaliação	42
16.1. Competências de Modelação	43
16.2. Competências de Implementação	43
16.3. Competências de Teste e Qualidade.....	43
16.4. Competências Comunicacionais.....	43

1. Título Geral da Avaliação Prática

Avaliação Prática Semestral, Desenvolvimento de Projectos em *C# Console Application* com Modelação Orientada a Objectos e Diagramas de Classes UML.

2. Enquadramento Académico e Justificativo Pedagógico

A presente proposta insere-se no contexto da Unidade Curricular de Programação Orientada a Objectos II com C#, ministrada no 3.º Ano da Licenciatura em Engenharia Informática. Esta unidade curricular tem como missão aprofundar e consolidar as competências técnicas e científicas dos estudantes no domínio da programação orientada a objectos, com particular enfoque na modelação de sistemas, na aplicação rigorosa dos princípios de encapsulamento, herança, polimorfismo e abstracção, e no desenvolvimento de aplicações robustas e bem estruturadas em linguagem C#.

A avaliação prática proposta constitui um elemento avaliativo de natureza integradora, permitindo aos estudantes demonstrar, num contexto aplicado e próximo da realidade profissional, o domínio dos conceitos teóricos previamente abordados nas aulas. A realização de projectos em grupos de três estudantes promove, adicionalmente, competências transversais de trabalho colaborativo, comunicação técnica e gestão partilhada de um produto de software.

A opção pelo formato de *C# Console Application*, sem recurso a interfaces gráficas, bases de dados externas ou *frameworks* avançadas, é uma decisão pedagógica deliberada: ao eliminar dependências tecnológicas complexas, privilegia-se a avaliação das competências nucleares de modelação orientada a objectos, *design* de classes, aplicação de padrões estruturais e qualidade do código-fonte. A elaboração obrigatória de diagramas de classes UML complementa este objectivo, avaliando a capacidade de abstracção e de comunicação técnica visual dos estudantes.

3. Objectivo Geral da Avaliação

A avaliação prática tem como objectivo geral aferir a capacidade dos estudantes para analisar um problema do mundo real, modelar uma solução orientada a objectos utilizando diagramas de classes UML, implementar essa solução em *C# Console Application* com rigor técnico, testar e validar o sistema desenvolvido, e comunicar os resultados numa apresentação pública fundamentada.

4. Objectivos Específicos

- Identificar e decompor os requisitos funcionais e não funcionais de um problema proposto.
- Elaborar um diagrama de classes UML completo e conforme com a notação *standard*, representando as entidades, atributos, métodos, relações, multiplicidades e modificadores de acesso.
- Implementar o modelo UML em C#, respeitando rigorosamente os princípios de encapsulamento, herança, polimorfismo, abstracção e composição.
- Aplicar colecções genéricas, tratamento de excepções personalizadas e validação robusta de dados de entrada.
- Organizar o código-fonte em pastas e namespaces segundo uma arquitectura de separação de responsabilidades.
- Elaborar documentação técnica clara e completa, incluindo relatório, análise de requisitos, descrição de algoritmos e evidências de teste.
- Apresentar e defender publicamente o projecto, demonstrando domínio técnico individual.

5. Competências Técnicas e Científicas a Avaliar

5.1. Competências de Análise e Modelação

- Análise de requisitos e especificação de funcionalidades
- Modelação orientada a objectos com diagramas de classes UML
- Identificação correcta de relações entre classes (associação, agregação, composição, herança, realização)

5.2. Competências de Implementação

- Encapsulamento e aplicação correcta de modificadores de acesso em C#
- Herança e polimorfismo (sobrescrita e sobrecarga de métodos)
- Classes abstractas e *interfaces*
- Composição e agregação entre objectos
- Colecções genéricas (*List<T>*, *Dictionary<K,V>*, *Queue<T>*, *Stack<T>*)
- Tratamento de excepções com classes personalizadas derivadas de *Exception*
- Validação de dados com lógica de negócio encapsulada

5.3. Competências de Qualidade de Software

- Organização do código em namespaces e pastas com separação de responsabilidades
- Legibilidade, convenções de nomenclatura e comentários XML
- Elaboração e execução de testes funcionais

5.4. Competências Comunicacionais e Documentais

- Elaboração de relatório técnico em linguagem formal e academicamente correcta
- Apresentação oral estruturada e defesa técnica do projecto

6. Regras de Organização dos Grupos

1. Os grupos são constituídos por exactamente três (3) estudantes, matriculados na Unidade Curricular no semestre lectivo em curso.
2. A constituição dos grupos é realizada no prazo estipulado pelo docente, podendo ser feita por escolha dos estudantes ou por atribuição docente.
3. A atribuição de projectos é efectuada por sorteio ou por escolha ordenada dos grupos, conforme decisão do docente responsável.
4. Cada projecto é atribuído a um único grupo. Não são permitidos projectos repetidos entre grupos da mesma turma.
5. A divisão de trabalho dentro do grupo deve ser equitativa e documentada no relatório técnico, com identificação das contribuições individuais de cada membro.
6. A avaliação individual dentro do grupo poderá ser diferenciada com base no desempenho demonstrado na apresentação pública e na defesa técnica.

7. Condições Técnicas Obrigatórias

- Linguagem de programação: C# (.NET 6 ou superior)
- Tipo de projecto: *Console Application*, sem interfaces gráficas (*WinForms*, *WPF*, *MAUI* ou equivalentes)
- Sem utilização de bases de dados externas (*SQL Server*, *SQLite*, *MongoDB* ou similares), os dados são geridos exclusivamente em memória com colecções genéricas
- Sem utilização de bibliotecas de terceiros (*NuGet packages* externos), apenas a biblioteca *standard* do .NET

- Sem utilização de *frameworks* de acesso a dados (*Entity Framework*, *Dapper* ou similares)
- Ambiente de desenvolvimento recomendado: Visual Studio 2026 ou Visual Studio Code com extensão C#
- Controlo de versões: utilização recomendada de Git com repositório privado, a ser entregue juntamente com o relatório
- O diagrama UML deve ser elaborado com recurso a uma ferramenta adequada (*StarUML*, *draw.io*, *Visual Paradigm*, *Astah* ou equivalente) e entregue em formato imagem (PNG ou PDF) e em formato editável

8. Lista de 34 Projectos

A seguir apresenta-se a lista completa dos trinta e quatro projectos disponíveis. Cada projecto é descrito de forma detalhada, contemplando o contexto do problema, o objectivo do sistema, as principais funcionalidades, as classes sugeridas, as relações entre classes, os conceitos de POO a aplicar, as excepções personalizadas previstas, os requisitos mínimos de teste e as sugestões para o diagrama de classes UML.

Projecto 1: Sistema de Gestão de Biblioteca

Sistema de Gestão de Biblioteca	
Contexto	Uma biblioteca municipal necessita de um sistema informatizado para controlar o acervo de livros, as inscrições de leitores e os empréstimos de obras.
Objectivo	Desenvolver um sistema orientado a objectos que permita gerir o acervo bibliográfico, registar leitores e controlar o ciclo de empréstimo e devolução de livros.
Funcionalidades	• Registo, edição e remoção de livros (título, ISBN, autor, ano, exemplares) • Cadastro e gestão de leitores com validação de dados • Realização e registo de empréstimos com data prevista de devolução • Processamento de devoluções com cálculo automático de multas por atraso • Pesquisa de livros por título, autor ou ISBN • Listagem de empréstimos em aberto e histórico por leitor
Classes	Livro, Leitor, Emprestimo, Catalogo, GestorBiblioteca

Relações	GestorBiblioteca agrega Catalogo; Emprestimo associa Livro e Leitor; Catalogo compõe colecção de Livro.
Conceitos POO	Encapsulamento, composição, colecções genéricas (List<T>, Dictionary<K,V>), interfaces de pesquisa, separação de responsabilidades.
Excepções	LivroNaoEncontradoException, LimiteEmprestimoExcedidoException, LivroIndisponivelException.
Testes	Adicionar livros; realizar empréstimo; tentar empréstimo sem exemplares; registar devolução com atraso; pesquisar livro inexistente.
Sugestões UML	Classes com atributos privados, propriedades públicas, métodos de negócio; associação entre Emprestimo–Livro–Leitor; composição Catalogo–Livro.

Projecto 2: Sistema de Gestão Escolar

Sistema de Gestão Escolar	
Contexto	Uma escola secundária precisa de um sistema para gerir turmas, disciplinas, docentes, alunos e lançamento de notas por período lectivo.
Objectivo	Modelar e implementar um sistema que centralize a gestão pedagógica e académica de uma escola, incluindo pautas e cálculo de médias.
Funcionalidades	• Registo de alunos e docentes com dados pessoais e profissionais • Criação e gestão de turmas e disciplinas • Atribuição de docentes a disciplinas e turmas • Lançamento de notas por avaliação e cálculo de médias • Geração de pautas por turma e disciplina • Verificação de aprovação ou reprovação por aluno
Classes	Pessoa, Aluno, Docente, Turma, Disciplina, Nota, GestorEscolar
Relações	Aluno e Docente herdam Pessoa; Turma agrega Aluno e Disciplina; Nota associa Aluno e Disciplina.
Conceitos POO	Herança (Pessoa → Aluno, Docente), polimorfismo em método ObterDetalhes(), classes abstractas, colecções genéricas.

Excepções	AlunoJaMatriculadoException, NotaInvalidaException, TurmaLotadaException.
Testes	Cadastrar aluno e docente; criar turma; lançar nota válida e inválida; calcular média; verificar aprovação.
Sugestões UML	Hierarquia de herança Pessoa; associações Turma–Aluno, Turma–Disciplina; composição GestorEscolar.

Projecto 3: Sistema de Controlo de Stock para Supermercado

Sistema de Controlo de Stock para Supermercado	
Contexto	Um supermercado de médio porte necessita de um sistema para controlar o inventário de produtos, registar entradas e saídas de stock e emitir alertas de ruptura.
Objectivo	Implementar um sistema de controlo de inventário que permita gerir produtos por categoria, registar movimentações e emitir alertas automáticos de stock mínimo.
Funcionalidades	• Cadastro de produtos com código, nome, categoria, preço de custo e preço de venda • Registo de entradas de stock (compras ao fornecedor) • Registo de saídas de stock (vendas ou consumo interno) • Alerta automático quando o stock atinge o nível mínimo definido • Relatório de produtos com stock em ruptura • Cálculo do valor total do inventário
Classes	Produto, Categoria, Movimentacao, Stock, GestorInventario
Relações	Produto pertence a Categoria; Stock compõe Produto; Movimentacao associa Produto.
Conceitos POO	Encapsulamento, composição, enumerações (TipoMovimentacao), interfaces (IAlertavel), colecções genéricas.
Excepções	StockInsuficienteException, ProdutoNaoEncontradoException, QuantidadeInvalidaException.
Testes	Adicionar produto; registar entrada; tentar saída com stock insuficiente; verificar alerta de mínimo; calcular valor total.
Sugestões UML	Composição Stock–Produto; associação Movimentacao–Produto; enum TipoMovimentacao; interface IAlertavel.

Projecto 4: Sistema de Agendamento Médico

Sistema de Agendamento Médico	
Contexto	Uma clínica médica privada necessita de um sistema de marcação de consultas, gestão de médicos e histórico clínico simplificado dos pacientes.
Objectivo	Desenvolver um sistema que permita marcar, alterar e cancelar consultas, gerir a disponibilidade de médicos e manter o histórico de atendimentos.
Funcionalidades	• Cadastro de médicos com especialidade e horários disponíveis • Cadastro de pacientes com dados pessoais e contactos • Marcação, alteração e cancelamento de consultas • Verificação de disponibilidade antes de marcar consulta • Registo do histórico de consultas por paciente • Listagem de consultas do dia para cada médico
Classes	Pessoa, Medico, Paciente, Consulta, Agenda, GestorClinica
Relações	Medico e Paciente herdam Pessoa; Consulta associa Medico e Paciente; Agenda agrega Consulta.
Conceitos POO	Herança, polimorfismo, composição Agenda–Consulta, interface IAgendavel, enumeração EstadoConsulta.
Excepções	MedicoIndisponivelException, ConsultaNaoEncontradaException, AgendamentoDuplicadoException.
Testes	Marcar consulta; tentar marcar em horário ocupado; cancelar consulta; listar consultas do dia; registar histórico.
Sugestões UML	Herança Pessoa; enum EstadoConsulta; associações Consulta–Medico–Paciente; composição Agenda.

Projecto 5: Sistema de Gestão de Recursos Humanos

Sistema de Gestão de Recursos Humanos	
Contexto	Uma empresa de médio porte necessita de um sistema para gerir os dados dos seus colaboradores, contratos de trabalho, departamentos e folha de salários.

Objectivo	Modelar e implementar um sistema de RH que centralize a gestão de colaboradores, calcule salários com descontos e adicione pagamentos à folha mensal.
Funcionalidades	• Registo de colaboradores com dados pessoais e contratuais • Gestão de departamentos e atribuição de colaboradores • Cálculo de salário líquido com descontos de IRT e segurança social • Processamento mensal da folha de pagamento • Registo de ausências, faltas e horas extraordinárias • Relatório da folha de salários por departamento
Classes	Colaborador, Contrato, Departamento, FolhaSalario, Desconto, GestorRH
Relações	Colaborador tem Contrato; Colaborador pertence a Departamento; FolhaSalario agrega Colaborador.
Conceitos POO	Herança (se houver tipos de colaborador), composição, interfaces (ICalculavel), colecções genéricas.
Excepções	ColaboradorNaoEncontradoException, SalarioInvalidoException, DepartamentoInexistenteException.
Testes	Cadastrar colaborador; calcular salário; processar folha; registar ausência; listar por departamento.
Sugestões UML	Classes Colaborador, Contrato, Departamento; composição FolhaSalario–Colaborador; interface ICalculavel.

Projecto 6: Sistema de Gestão de Hotel

Sistema de Gestão de Hotel	
Contexto	Um hotel de três estrelas necessita de um sistema para gerir quartos, reservas, check-in, check-out e facturação de estadias.
Objectivo	Implementar um sistema hoteleiro que permita gerir a disponibilidade de quartos, realizar reservas e processar a facturação automática das estadias.
Funcionalidades	• Cadastro de quartos com tipo, capacidade e preço por noite • Realização e cancelamento de reservas com validação de datas

	• Processo de check-in e check-out com cálculo automático do valor da estadia • Listagem de quartos disponíveis para um intervalo de datas • Geração de factura detalhada com serviços adicionais • Histórico de estadias por cliente
Classes	Quarto, Cliente, Reserva, Estadia, Factura, GestorHotel
Relações	Reserva associa Quarto e Cliente; Estadia herda ou compõe Reserva; Factura agrega itens de serviço.
Conceitos POO	Encapsulamento, composição, enumerações (TipoQuarto, EstadoReserva), colecções genéricas, separação de responsabilidades.
Excepções	QuartoIndisponivelException, DataInvalidaException, ReservaNaoEncontradaException.
Testes	Criar reserva; verificar disponibilidade; fazer check-in; fazer check-out com cálculo; cancelar reserva.
Sugestões UML	Enum TipoQuarto; enum EstadoReserva; associações Reserva–Quarto–Cliente; composição Factura.

Projecto 7: Sistema de Gestão de Transportadora

Sistema de Gestão de Transportadora	
Contexto	Uma empresa de transporte rodoviário precisa de gerir a sua frota de veículos, os motoristas, as rotas e os registos de viagens realizadas.
Objectivo	Desenvolver um sistema que permita controlar a frota, associar motoristas a veículos, registar viagens e calcular custos operacionais.
Funcionalidades	• Cadastro de veículos com matrícula, tipo, capacidade e estado • Cadastro de motoristas com carta de condução e habilitações • Criação e gestão de rotas com origem, destino e distância • Registo de viagens com associação de veículo, motorista e rota • Cálculo do custo da viagem com base em consumo e distância • Relatório de manutenções programadas por veículo
Classes	Veiculo, Motorista, Rota, Viagem, Manutencao, GestorTransportadora

Relações	Viagem associa Veiculo, Motorista e Rota; Manutencao compõe Veiculo; GestorTransportadora agrega colecções.
Conceitos POO	Encapsulamento, composição, enumerações (TipoVeiculo, EstadoVeiculo), interfaces (IRegistavel), colecções genéricas.
Excepções	VeiculoIndisponivelException, MotoristaInabilitadoException, RotaInexistenteException.
Testes	Cadastrar veículo; registar viagem; calcular custo; listar viagens de um motorista; verificar manutenção.
Sugestões UML	Enum TipoVeiculo; composição Manutencao–Veiculo; associações Viagem–Veiculo–Motorista–Rota.

Projecto 8: Sistema de Gestão Bancária Simplificada

Sistema de Gestão Bancária Simplificada	
Contexto	Um banco pretende um sistema para gerir contas bancárias de clientes, efectuar operações de depósito, levantamento e transferência, e manter extractos.
Objectivo	Implementar um sistema que modele diferentes tipos de contas bancárias, suporte operações financeiras básicas e registe movimentos no extracto.
Funcionalidades	• Criação de conta corrente e conta poupança com dados do titular • Operações de depósito, levantamento e transferência entre contas • Verificação de saldo disponível antes de movimentos a débito • Cálculo automático de juros para conta poupança • Emissão de extracto de movimentos por intervalo de datas • Bloqueio e desbloqueio de contas
Classes	Conta, ContaCorrente, ContaPoupanca, Cliente, Movimento, GestorBanco
Relações	ContaCorrente e ContaPoupanca herdam Conta; Movimento compõe Conta; Cliente agrega Conta.
Conceitos POO	Herança abstracta, polimorfismo (Depositar, Levantar), interfaces (IContaBancaria), encapsulamento rigoroso.

Excepções	SaldoInsuficienteException, ContaBloqueadaException, TransferenciaNaoPermitidaException.
Testes	Criar conta; depositar; levantar com saldo suficiente e insuficiente; transferir; gerar extracto.
Sugestões UML	Classe abstracta Conta; herança ContaCorrente, ContaPoupanca; interface IContaBancaria; composição Movimento.

Projecto 9: Sistema de Gestão de Farmácia

Sistema de Gestão de Farmácia	
Contexto	Uma farmácia comunitária necessita de um sistema para gerir o seu stock de medicamentos, registar vendas, controlar prazos de validade e gerir fornecedores.
Objectivo	Desenvolver um sistema que permita controlar o inventário de medicamentos com validade, processar vendas com receitas e emitir alertas de validade.
Funcionalidades	• Cadastro de medicamentos com nome, DCI, lote, validade e stock • Registo de vendas com ou sem prescrição médica • Verificação de prazo de validade no momento da venda • Alerta de medicamentos próximos do vencimento • Gestão de fornecedores e registo de encomendas • Relatório de movimentos de stock por período
Classes	Medicamento, Venda, ItemVenda, Fornecedor, Encomenda, GestorFarmacia
Relações	Venda agrega ItemVenda; ItemVenda associa Medicamento; Encomenda associa Fornecedor e Medicamento.
Conceitos POO	Composição, colecções genéricas, interfaces (IVerificavelValidade), enumeração (TipoMedicamento).
Excepções	MedicamentoVencidoException, StockInsuficienteException, ReceitaObrigatoriaException.
Testes	Registar medicamento; efectuar venda; tentar vender medicamento vencido; alerta de validade; relatório de stock.

Sugestões UML	Composição Venda–ItemVenda; associação ItemVenda–Medicamento; interface IVerificavelValidade.
----------------------	---

Projecto 10: Sistema de Gestão de Academia de Ginástica

Sistema de Gestão de Academia de Ginástica	
Contexto	Uma academia de ginástica pretende gerir as inscrições de sócios, os planos de treino, os pagamentos mensais e a frequência nas aulas colectivas.
Objectivo	Implementar um sistema que controle sócios, planos de mensalidade, aulas disponíveis, inscrições e o registo de frequência.
Funcionalidades	• Cadastro de sócios com dados pessoais e plano de mensalidade • Registo e gestão de planos de treino por tipo (musculação, cardio, misto) • Inscrição de sócios em aulas colectivas com controlo de vagas • Registo de pagamentos mensais e verificação de situação • Alerta automático de pagamentos em atraso • Relatório de frequência por sócio e por aula
Classes	Socio, PlanoMensalidade, AulaColectiva, Pagamento, Inscricao, GestorAcademia
Relações	Socio tem PlanoMensalidade; Inscricao associa Socio e AulaColectiva; Pagamento compõe Socio.
Conceitos POO	Composição, colecções genéricas, interfaces (INotificavel), enum (EstadoPagamento, TipoPlano).
Excepções	AulaLotadaException, PagamentoEmAtrasoException, SocioNaoEncontradoException.
Testes	Inscrever sócio; registar pagamento; tentar inscrição em aula lotada; verificar atraso; emitir relatório.
Sugestões UML	Composição Pagamento–Socio; associação Inscricao–AulaColectiva; enum EstadoPagamento; interface INotificavel.

Projecto 11: Sistema de Gestão de Concessionária Automóvel

Sistema de Gestão de Concessionária Automóvel

Contexto	Uma concessionária de veículos novos e usados necessita de um sistema para gerir o inventário de viaturas, registar vendas e controlar comissões dos vendedores.
Objectivo	Modelar e implementar um sistema que permita gerir o parque automóvel, processar vendas com financiamento opcional e calcular comissões.
Funcionalidades	• Cadastro de veículos novos e usados com dados técnicos e comerciais • Registo de clientes e histórico de contactos • Processamento de vendas com opção de pagamento a pronto ou financiado • Cálculo automático de comissão por vendedor • Controlo de test drives e visitas de clientes • Relatório de vendas por período e por vendedor
Classes	Veiculo, VeiculoNovo, VeiculoUsado, Cliente, Venda, Vendedor, GestorConcessionaria
Relações	VeiculoNovo e VeiculoUsado herdam Veiculo; Venda associa Veiculo, Cliente e Vendedor.
Conceitos POO	Herança, polimorfismo (calcular preço), interfaces (IComissionavel), colecções genéricas.
Excepções	VeiculoJaVendidoException, DescontoExcedidoException, ClienteNaoEncontradoException.
Testes	Cadastrar veículo; registar venda; calcular comissão; gerar relatório; verificar veículo já vendido.
Sugestões UML	Herança Veiculo–VeiculoNovo/Usado; interface IComissionavel; associações Venda.

Projecto 12: Sistema de Gestão de Restaurante

Sistema de Gestão de Restaurante	
Contexto	Um restaurante de serviço completo necessita de um sistema para gerir mesas, ementa, pedidos de clientes, cozinha e facturação.
Objectivo	Desenvolver um sistema que permita gerir o atendimento ao cliente, registar pedidos, comunicar com a cozinha e emitir facturas detalhadas.
Funcionalidades	• Cadastro e gestão de mesas com estado (livre, ocupada, reservada) • Gestão da ementa com pratos, bebidas e preços

	• Registo de pedidos associados a mesas com itens e quantidades • Actualização do estado do pedido (pendente, em preparo, servido) • Cálculo automático da factura com IVA • Relatório de vendas e pratos mais pedidos do dia
Classes	Mesa, ItemEmenta, Pedido, ItemPedido, Factura, GestorRestaurante
Relações	Pedido agrega ItemPedido; ItemPedido associa ItemEmenta; Factura compõe Pedido; Mesa associa Pedido.
Conceitos POO	Composição, enumerações (EstadoMesa, EstadoPedido), colecções genéricas, interfaces (IFacturavel).
Excepções	MesaOcupadaException, ItemIndisponivelException, PedidoJaFechadoException.
Testes	Abrir pedido; adicionar itens; fechar pedido; calcular factura; tentar pedido em mesa ocupada.
Sugestões UML	Enum EstadoMesa; composição Factura–Pedido–ItemPedido; interface IFacturavel; associação ItemPedido–ItemEmenta.

Projecto 13: Sistema de Gestão de Condomínio

Sistema de Gestão de Condomínio	
Contexto	A administração de um condomínio residencial precisa de um sistema para gerir fracções, condóminos, cotas mensais, despesas comuns e assembleias.
Objectivo	Implementar um sistema que controle as fracções do condomínio, gere cotas, registre pagamentos e suporte a convocação de assembleias.
Funcionalidades	• Cadastro de fracções com identificação, área e proprietário • Geração automática de cotas mensais por fracção • Registo de pagamentos e controlo de dívidas • Registo de despesas comuns e apuramento de saldos • Convocação e registo de actas de assembleias • Relatório da situação financeira do condomínio
Classes	Fraccao, Condomino, Cota, Pagamento, Despesa, Assembleia, GestorCondominio

Relações	Fracao pertence a Condomino; Cota compõe Fracao; Pagamento associa Cota e Condomino.
Conceitos POO	Composição, colecções genéricas, interfaces (IGeravelCota), enum (EstadoCota), encapsulamento.
Excepções	PagamentoEmAtrasoException, FracaoNaoEncontradaException, AssembleiaJaConvocadaException.
Testes	Cadastrar fracção; gerar cota; registar pagamento; calcular dívida; listar moradores em atraso.
Sugestões UML	Composição Cota–Fracao; associação Pagamento–Cota; enum EstadoCota; interface IGeravelCota.

Projecto 14: Sistema de Gestão de Companhia Aérea

Sistema de Gestão de Companhia Aérea	
Contexto	Uma companhia aérea regional precisa de um sistema para gerir os seus voos, aeronaves, tripulação, reservas de passageiros e check-in.
Objectivo	Desenvolver um sistema que permita gerir a operação de voos, efectuar reservas, controlar a ocupação das aeronaves e processar o embarque.
Funcionalidades	• Cadastro de aeronaves com tipo, capacidade e classe de assentos • Criação e gestão de voos com origem, destino, data e horário • Reserva e cancelamento de bilhetes com selecção de assento • Processo de check-in com confirmação de embarque • Cálculo do preço do bilhete por classe e antecedência • Listagem de passageiros por voo
Classes	Aeronave, Voo, Passageiro, Reserva, Bilhete, GestorCompanhia
Relações	Voo associa Aeronave; Reserva associa Voo e Passageiro; Bilhete compõe Reserva.
Conceitos POO	Composição, herança (se houver tipos de assento), enum (ClasseVoo, EstadoReserva), colecções genéricas.
Excepções	AssentoOcupadoException, VooLotadoException, CheckInForaDoPrazoException.

Testes	Criar voo; reservar bilhete; seleccionar assento; fazer check-in; cancelar reserva; listar passageiros.
Sugestões UML	Enum ClasseVoo, EstadoReserva; composição Bilhete–Reserva; associações Voo–Aeronave–Passageiro.

Projecto 15: Sistema de Gestão de Eventos e Conferências

Sistema de Gestão de Eventos e Conferências	
Contexto	Uma empresa de organização de eventos necessita de um sistema para gerir eventos, vendas de bilhetes, oradores, patrocinadores e a agenda do evento.
Objectivo	Modelar e implementar um sistema que suporte o ciclo completo de organização de um evento, desde a criação até ao controlo de presenças.
Funcionalidades	• Criação e gestão de eventos com local, data, capacidade e tipo • Gestão de sessões, painéis e oradores por evento • Venda de bilhetes com categorias de preço (geral, VIP, estudante) • Controlo de acesso e registo de presenças por QR simulado • Gestão de patrocinadores com valores de patrocínio • Relatório de ocupação e receita por evento
Classes	Evento, Sessao, Orador, Bilhete, Participante, Patrocinador, GestorEventos
Relações	Evento agrega Sessao; Sessao associa Orador; Bilhete associa Participante e Evento.
Conceitos POO	Composição, colecções genéricas, enum (CategoriaIngresso, TipoEvento), interfaces (INotificavel).
Excepções	EventoLotadoException, BilheteInvalidoException, DataConflituosaException.
Testes	Criar evento; adicionar sessão; vender bilhete; verificar capacidade; registar presença; gerar relatório.
Sugestões UML	Composição Evento–Sessao; associação Bilhete–Participante; enum CategoriaIngresso; interface INotificavel.

Projecto 16: Sistema de Gestão de Loja de Informática

Sistema de Gestão de Loja de Informática	
Contexto	Uma loja de equipamentos e periféricos informáticos precisa de um sistema para gerir o seu inventário, processar vendas, gerir garantias e controlar devoluções.
Objectivo	Implementar um sistema orientado a objectos para gestão comercial de uma loja de informática com controlo de garantias e pós-venda.
Funcionalidades	• Cadastro de produtos por categoria (computadores, periféricos, componentes) • Gestão de stock com alertas de nível mínimo • Processamento de vendas com emissão de recibo e garantia • Registo e tratamento de devoluções dentro do prazo de garantia • Gestão de fornecedores e encomendas de reposição • Relatório de vendas diárias, semanais e mensais
Classes	Produto, Categoria, Venda, ItemVenda, Garantia, Fornecedor, GestorLoja
Relações	Produto tem Categoria; Venda agrega ItemVenda; Garantia compõe ItemVenda; Produto associa Fornecedor.
Conceitos POO	Composição, colecções genéricas, enum (TipoProduto, EstadoGarantia), interface (IDevolvivel).
Excepções	GarantiaExpiradaException, StockInsuficienteException, DevolucaoForaDoPrazoException.
Testes	Registar produto; processar venda; emitir garantia; registar devolução válida e fora do prazo; alerta de stock.
Sugestões UML	Composição Garantia–ItemVenda; interface IDevolvivel; enum TipoProduto; associação Produto–Fornecedor.

Projecto 17: Sistema de Gestão de Clínica Veterinária

Sistema de Gestão de Clínica Veterinária	
Contexto	Uma clínica veterinária precisa de gerir os seus pacientes animais, os tutores, as consultas, os tratamentos e o stock de medicamentos veterinários.

Objectivo	Desenvolver um sistema que permita gerir o ciclo completo de atendimento veterinário, desde o registo do animal até ao histórico de tratamentos.
Funcionalidades	• Cadastro de animais (espécie, raça, idade, tutor) e tutores • Marcação e registo de consultas veterinárias • Prescrição de medicamentos e tratamentos • Controlo do calendário de vacinas por animal • Stock de medicamentos veterinários com validade • Histórico clínico completo por animal
Classes	Animal, Tutor, Medico, Consulta, Tratamento, Vacina, GestorClinicaVet
Relações	Animal pertence a Tutor; Consulta associa Animal e Medico; Tratamento compõe Consulta; Vacina agrega Animal.
Conceitos POO	Composição, colecções genéricas, herança (se houver tipos de animal), enum (EspecieAnimal), interfaces.
Excepções	AnimalNaoEncontradoException, VacinaVencidaException, ConsultaNaoAgendadaException.
Testes	Cadastrar animal e tutor; marcar consulta; prescrever tratamento; registar vacina; verificar histórico.
Sugestões UML	Composição Tratamento–Consulta; associação Consulta–Animal–Medico; enum EspecieAnimal.

Projecto 18: Sistema de Gestão de Projectos de Software

Sistema de Gestão de Projectos de Software	
Contexto	Uma empresa de desenvolvimento de software necessita de um sistema para gerir projectos, equipas, tarefas, sprints e o progresso de entregas.
Objectivo	Modelar e implementar um sistema de gestão de projectos ágeis que suporte a criação de tarefas, atribuição a equipas e controlo do ciclo de vida dos projectos.
Funcionalidades	• Criação e gestão de projectos com datas, orçamento e estado • Gestão de equipas e membros com papéis definidos

	• Criação e atribuição de tarefas com prioridade e prazo • Organização de sprints com backlog e objectivos • Registo do progresso e alteração de estado de tarefas • Relatório de progresso por projecto e por sprint
Classes	Projecto, Equipa, Membro, Tarefa, Sprint, GestorProjectos
Relações	Projecto agrega Sprint; Sprint agrega Tarefa; Tarefa associa Membro; Equipa agrega Membro.
Conceitos POO	Composição, colecções genéricas, enum (EstadoTarefa, Prioridade, PapelMembro), interfaces (IProgressavel).
Excepções	TarefaNaoAtribuidaException, SprintFechadoException, MembroNaoEncontradoException.
Testes	Criar projecto; adicionar tarefa; atribuir a membro; avançar estado; criar sprint; gerar relatório.
Sugestões UML	Composição Projecto–Sprint–Tarefa; enum EstadoTarefa, Prioridade; interface IProgressavel.

Projecto 19: Sistema de Gestão de Compras Online (E-commerce Simplificado)

Sistema de Gestão de Compras Online (E-commerce Simplificado)	
Contexto	Uma pequena loja online necessita de um sistema para gerir o catálogo de produtos, registar encomendas de clientes e controlar o estado das expedições.
Objectivo	Implementar um sistema de e-commerce simplificado que suporte a navegação no catálogo, criação de encomendas, gestão de carrinho e controlo de expedição.
Funcionalidades	• Catálogo de produtos com pesquisa por nome e categoria • Gestão do carrinho de compras por sessão de cliente • Criação de encomendas com validação de stock • Cálculo do valor total com taxa de envio • Actualização do estado da encomenda (pendente, confirmada, expedida, entregue) • Histórico de encomendas por cliente

Classes	Produto, Cliente, Carrinho, Encomenda, ItemEncomenda, GestorEcommerce
Relações	Carrinho agrega ItemEncomenda; Encomenda agrega ItemEncomenda; ItemEncomenda associa Produto.
Conceitos POO	Composição, colecções genéricas, enum (EstadoEncomenda), interface (IProcessavelEncomenda).
Excepções	ProdutoEsgotadoException, CarrinhoVazioException, EncomendaJaProcessadaException.
Testes	Adicionar ao carrinho; criar encomenda; alterar estado; calcular total; tentar encomenda sem stock.
Sugestões UML	Composição Carrinho–ItemEncomenda; enum EstadoEncomenda; interface IProcessavelEncomenda.

Projecto 20: Sistema de Gestão de Eleições Estudantis

Sistema de Gestão de Eleições Estudantis	
Contexto	Uma universidade pretende digitalizar o processo de eleições para órgãos estudantis, permitindo o registo de candidaturas, votação electrónica e apuramento de resultados.
Objectivo	Desenvolver um sistema que suporte o processo eleitoral académico, garantindo a integridade do voto, o apuramento transparente e a publicação de resultados.
Funcionalidades	• Registo de eleitores (estudantes) com validação de elegibilidade • Submissão e validação de candidaturas por lista ou individual • Processo de votação com verificação de voto único por eleitor • Apuramento automático de resultados após encerramento • Publicação e visualização dos resultados finais • Relatório estatístico de participação eleitoral
Classes	Eleitor, Candidatura, Eleicao, Voto, ApuramentoResultados, GestorEleicoes
Relações	Eleicao agrega Candidatura e Voto; Voto associa Eleitor e Candidatura; ApuramentoResultados compõe Eleicao.

Conceitos POO	Composição, colecções genéricas, enum (EstadoEleicao), interface (IApuravel), encapsulamento rigoroso.
Excepções	VotoDuplicadoException, PeriodoEleitoralEncerradoException, CandidaturaInvalidaException.
Testes	Registar eleitor; submeter candidatura; votar; tentar votar duas vezes; apurar resultados; gerar relatório.
Sugestões UML	Composição Eleicao–Voto–Candidatura; enum EstadoEleicao; interface IApuravel; encapsulamento forte.

Projecto 21: Sistema de Gestão de Centro de Explicações

Sistema de Gestão de Centro de Explicações	
Contexto	Um centro de explicações privado precisa de um sistema para gerir os seus alunos, professores, disciplinas, horários de aulas e pagamentos de mensalidades.
Objectivo	Implementar um sistema que suporte o funcionamento administrativo e pedagógico de um centro de explicações, incluindo gestão de horários e cobranças.
Funcionalidades	• Registo de alunos com dados pessoais e disciplinas inscritas • Cadastro de professores com habilitações e disciplinas que leccionam • Criação de horários com associação de professor, aluno e sala • Registo de presenças por aula • Geração de mensalidades e controlo de pagamentos • Relatório de assiduidade por aluno
Classes	Aluno, Professor, Disciplina, Horario, Presenca, Mensalidade, GestorCentro
Relações	Horario associa Professor, Aluno e Disciplina; Presenca compõe Horario; Mensalidade compõe Aluno.
Conceitos POO	Composição, colecções genéricas, enum (EstadoPresenca, EstadoPagamento), interface (IAgendavel).
Excepções	HorarioConflituosoException, MensalidadeEmAtrasoException, AlunoNaolInscritoException.

Testes	Registar aluno; criar horário; marcar presença; gerar mensalidade; registar pagamento; relatório de assiduidade.
Sugestões UML	Composição Mensalidade–Aluno; associação Horário–Professor–Aluno–Disciplina; enum EstadoPresenca.

Projecto 22: Sistema de Gestão de Cooperativa Agrícola

Sistema de Gestão de Cooperativa Agrícola	
Contexto	Uma cooperativa agrícola necessita de um sistema para gerir os sócios cooperativistas, os produtos agrícolas entregues, os preços de comercialização e as distribuições de lucros.
Objectivo	Desenvolver um sistema que controle as entregas de produtos pelos cooperativistas, o valor de comercialização e o apuramento de lucros a distribuir.
Funcionalidades	• Cadastro de cooperativistas com quota de participação • Registo de produtos agrícolas entregues por sócio e período • Definição de preços de comercialização por produto e época • Cálculo do valor apurado por sócio com base nas entregas • Distribuição de lucros proporcionalmente às quotas • Relatório de produção e comercialização por período
Classes	Cooperativista, ProdutoAgricultura, Entrega, Comercializacao, DistribuicaoLucro, GestorCooperativa
Relações	Entrega associa Cooperativista e ProdutoAgricultura; Comercializacao agrega Entrega; DistribuicaoLucro compõe Cooperativista.
Conceitos POO	Composição, colecções genéricas, interfaces (ICalculavelLucro), enum (EpocaAgricultura).
Excepções	CooperativistaNaoEncontradoException, QuantidadeInvalidaException, PeriodoEncerradoException.
Testes	Cadastrar cooperativista; registar entrega; calcular valor apurado; distribuir lucros; relatório de produção.
Sugestões UML	Composição DistribuicaoLucro–Cooperativista; associação Entrega–ProdutoAgricultura; interface ICalculavelLucro.

Projecto 23: Sistema de Gestão de Museu

Sistema de Gestão de Museu	
Contexto	Um museu nacional pretende um sistema para gerir o seu acervo de peças, as exposições temporárias e permanentes, os visitantes e os bilhetes.
Objectivo	Modelar e implementar um sistema de gestão museológica que permita controlar o acervo, organizar exposições e gerir a venda de bilhetes.
Funcionalidades	• Cadastro de peças com descrição, período histórico, origem e estado de conservação • Organização de exposições com curadoria e peças associadas • Venda de bilhetes com categorias de visitante (adulto, criança, sénior) • Registo de visitas guiadas com grupo e guia responsável • Controlo de empréstimos de peças para outros museus • Relatório de visitação por período e exposição
Classes	Peca, Exposicao, Visitante, Bilhete, VisitaGuiada, GestorMuseu
Relações	Exposicao agrega Peca; Bilhete associa Visitante; VisitaGuiada associa Exposicao.
Conceitos POO	Composição, colecções genéricas, enum (CategoriaBilhete, EstadoConservacao), interface (IExibivel).
Excepções	PecaNaoDisponivel, ExposicaoEncerradaException, CapacidadeExcedidaException.
Testes	Catalogar peça; criar exposição; vender bilhete; registar visita guiada; controlar empréstimo; relatório.
Sugestões UML	Composição Exposicao–Peca; associação Bilhete–Visitante; enum CategoriaBilhete; interface IExibivel.

Projecto 24: Sistema de Gestão de Empresa de Seguros

Sistema de Gestão de Empresa de Seguros	
Contexto	Uma seguradora necessita de um sistema para gerir apólices de seguro, segurados, sinistros e o processamento de indemnizações.

Objectivo	Implementar um sistema que suporte a gestão do ciclo de vida de apólices de seguro, desde a emissão até ao processamento de sinistros e pagamento de indemnizações.
Funcionalidades	• Emissão de apólices por tipo de seguro (automóvel, saúde, habitação) • Cadastro de segurados com dados pessoais e apólices activas • Registo e instrução de sinistros com documentação • Cálculo do valor da indemnização com base na cobertura • Controlo de pagamentos de prémios e renovações • Relatório de sinistros por tipo e período
Classes	Segurado, Apolice, TipoSeguro, Sinistro, Indemnizacao, GestorSeguros
Relações	Segurado tem Apolice; Apolice tem TipoSeguro; Sinistro associa Apolice; Indemnizacao compõe Sinistro.
Conceitos POO	Herança (por tipo de seguro), polimorfismo (calcularPremio), interface (ICobertura), enum (EstadoSinistro).
Excepções	ApoliceVencidaException, SinistroJaRegistadoException, IndemnizacaoExcedeLimiteException.
Testes	Emitir apólice; registar sinistro; calcular indemnização; verificar apólice vencida; relatório de sinistros.
Sugestões UML	Herança Apolice por tipo; composição Indemnizacao–Sinistro; interface ICobertura; enum EstadoSinistro.

Projecto 25: Sistema de Gestão de Centro de Saúde Público

Sistema de Gestão de Centro de Saúde Público	
Contexto	Um centro de saúde público necessita de um sistema para gerir o seu cadastro de utentes, triagem, consultas médicas, prescrições e referências hospitalares.
Objectivo	Desenvolver um sistema que apoie o processo de atendimento no centro de saúde, desde o registo do utente até à prescrição e eventual referência hospitalar.
Funcionalidades	• Cadastro de utentes com número de identificação e histórico de saúde • Sistema de triagem com classificação por prioridade

	• Marcação e registo de consultas com médico de família • Emissão de prescrições médicas com medicamentos e dosagens • Emissão de referências para especialidades hospitalares • Histórico clínico completo por utente
Classes	Utente, Medico, Consulta, Triagem, Prescricao, Referencia, GestorCentroSaude
Relações	Triagem precede Consulta; Consulta associa Utente e Medico; Prescricao compõe Consulta; Referencia compõe Consulta.
Conceitos POO	Composição, colecções genéricas, enum (NivelTriagem, TipoReferencia), interfaces (IRegistavelHistorico).
Excepções	UtenteSemTriagemException, MedicoNaoDisponivelException, PrescricaoInvalidaException.
Testes	Cadastrar utente; registar triagem; marcar consulta; emitir prescrição; emitir referência; consultar histórico.
Sugestões UML	Composição Prescricao–Consulta; enum NivelTriagem; interface IRegistavelHistorico; associações.

Projecto 26: Sistema de Gestão de Editora

Sistema de Gestão de Editora	
Contexto	Uma editora de livros académicos e técnicos precisa de um sistema para gerir os seus autores, manuscritos em revisão, publicações e distribuidores.
Objectivo	Implementar um sistema que controle o ciclo editorial completo, desde a submissão de um manuscrito até à publicação e distribuição.
Funcionalidades	• Cadastro de autores com dados pessoais e obras publicadas • Submissão e gestão de manuscritos com estado editorial • Processo de revisão por pares com registo de pareceres • Publicação de obras com ISBN, edição e tiragem • Gestão de distribuidores e encomendas de exemplares • Relatório de vendas e royalties por autor

Classes	Autor, Manuscrito, Obra, Revisor, Parecer, Distribuidor, GestorEditora
Relações	Manuscrito associa Autor; Parecer compõe Manuscrito; Obra herda ou transforma Manuscrito; Distribuidor agrega Obra.
Conceitos POO	Composição, colecções genéricas, enum (EstadoManuscrito, TipoObra), interface (IRevisavel).
Excepções	ManuscritoNaoEncontradoException, PareceNegativoException, ISBNDuplicadoException.
Testes	Submeter manuscrito; atribuir revisor; registar parecer; publicar obra; calcular royalties; gerar relatório.
Sugestões UML	Composição Parecer–Manuscrito; enum EstadoManuscrito; interface IRevisavel; associação Obra–Distribuidor.

Projecto 27: Sistema de Gestão de Câmara Municipal

Sistema de Gestão de Câmara Municipal	
Contexto	Uma câmara municipal necessita de um sistema para gerir pedidos de licenciamento, reclamações de munícipes, serviços prestados e arrecadação de taxas.
Objectivo	Desenvolver um sistema que suporte a gestão dos serviços administrativos de uma câmara municipal, priorizando o controlo de processos e a arrecadação de receitas.
Funcionalidades	• Registo de munícipes com dados de identificação • Submissão e acompanhamento de pedidos de licenciamento • Registo e tratamento de reclamações com prazos de resposta • Cálculo e emissão de guias de pagamento de taxas • Controlo de obras autorizadas por munícipe • Relatório de processos pendentes e resolvidos por período
Classes	Municipio, Pedido, Reclamacao, Licenca, Taxa, Servico, GestorCamara
Relações	Municipio submete Pedido; Pedido origina Licenca; Taxa compõe Pedido; Reclamacao associa Municipio.

Conceitos POO	Composição, colecções genéricas, enum (EstadoPedido, TipoLicenca), interface (IProcessavel).
Excepções	PedidoJaSubmetidoException, TaxaNaoPagaException, PrazoExcedidoException.
Testes	Registar munícipe; submeter pedido; emitir taxa; licenciar obra; registar reclamação; relatório de pendências.
Sugestões UML	Composição Taxa–Pedido; enum EstadoPedido, TipoLicenca; interface IProcessavel; associações.

Projecto 28: Sistema de Gestão de Concurso Público

Sistema de Gestão de Concurso Público	
Contexto	Um organismo público necessita de um sistema para gerir concursos de provimento de vagas, incluindo a publicação de vagas, inscrição de candidatos, provas e resultados.
Objectivo	Implementar um sistema que suporte o processo de concurso público, desde a abertura até à publicação dos resultados finais e ordenação dos candidatos.
Funcionalidades	• Criação e publicação de concursos com vagas, requisitos e prazos • Inscrição de candidatos com validação de requisitos • Registo de resultados por fase (prova escrita, oral, curricular) • Cálculo da classificação final ponderada • Ordenação de candidatos e publicação de listas preliminares e finais • Gestão de exclusões e recursos administrativos
Classes	Concurso, Candidato, Inscricao, Prova, Classificacao, GestorConcurso
Relações	Concurso agrega Inscricao; Inscricao associa Candidato; Classificacao compõe Inscricao; Prova associa Concurso.
Conceitos POO	Composição, colecções genéricas, enum (EstadoConcurso, TipoProva), interface (IClassificavel).
Excepções	PrazoInscricaoEncerradoException, RequisitoNaoSatisfeitoException, ClassificacaoJaAtribuidaException.

Testes	Criar concurso; inscrever candidato; registar nota; calcular classificação; ordenar lista; publicar resultado.
Sugestões UML	Composição Classificacao–Inscricao; enum EstadoConcurso; interface IClassificavel; associações.

Projecto 29: Sistema de Gestão de Agência de Viagens

Sistema de Gestão de Agência de Viagens	
Contexto	Uma agência de viagens necessita de um sistema para gerir pacotes turísticos, reservas de clientes, parceiros (hotéis, transportadoras) e pagamentos.
Objectivo	Desenvolver um sistema que permita gerir a oferta de pacotes turísticos, processar reservas de clientes e coordenar os serviços dos parceiros.
Funcionalidades	• Criação e gestão de pacotes turísticos com itinerário e preço • Cadastro de clientes e histórico de viagens • Reserva de pacotes com gestão de disponibilidade • Registo de parceiros (hotéis, transportadoras) e serviços • Processamento de pagamentos com parcelamento • Relatório de reservas e receitas por destino e período
Classes	Pacote, Cliente, Reserva, Parceiro, Servico, Pagamento, GestorAgencia
Relações	Pacote agrega Servico; Reserva associa Pacote e Cliente; Pagamento compõe Reserva; Servico associa Parceiro.
Conceitos POO	Composição, colecções genéricas, enum (EstadoReserva, TipoParceiro), interface (IReservavel).
Excepções	PacoteEsgotadoException, DataPartidaInvalidaException, PagamentoIncompletoException.
Testes	Criar pacote; reservar; processar pagamento; cancelar reserva; listar clientes por destino; relatório.
Sugestões UML	Composição Pagamento–Reserva; associação Reserva–Pacote–Cliente; enum EstadoReserva; interface IReservavel.

Projecto 30: Sistema de Gestão de Centro de Formação Profissional

Sistema de Gestão de Centro de Formação Profissional	
Contexto	Um centro de formação profissional precisa de um sistema para gerir os seus cursos, formadores, formandos, inscrições, avaliações e certificações.
Objectivo	Implementar um sistema que suporte o ciclo completo de formação profissional, desde a criação de cursos até à emissão de certificados de conclusão.
Funcionalidades	• Criação e gestão de cursos com plano curricular, duração e modalidade • Cadastro de formadores com habilitações e áreas de competência • Inscrição de formandos com validação de requisitos de acesso • Registo de presenças e avaliações por módulo • Cálculo da classificação final e verificação de aprovação • Emissão de certificados de conclusão para formandos aprovados
Classes	Curso, Formador, Formando, Inscricao, Modulo, Avaliacao, Certificado, GestorFormacao
Relações	Curso agrega Modulo; Inscricao associa Formando e Curso; Avaliacao compõe Modulo; Certificado compõe Inscricao.
Conceitos POO	Composição, colecções genéricas, enum (EstadoInscricao, Modalidade), interface (ICertificavel).
Excepções	FormandoJaInscritoException, AvaliacaoForaDoPrazoException, RequisitoAcessoNaoSatisfeitoException.
Testes	Criar curso; inscrever formando; lançar avaliação; calcular classificação; emitir certificado; listar aprovados.
Sugestões UML	Composição Avaliacao–Modulo; composição Certificado–Inscricao; enum Modalidade; interface ICertificavel.

Projecto 31: Sistema de Gestão de Rede de Postos de Combustível

Sistema de Gestão de Rede de Postos de Combustível
--

Contexto	Uma empresa petrolífera angolana opera uma rede de postos de combustível em diversas províncias e necessita de um sistema para gerir os seus postos, tanques, abastecimentos, funcionários e facturação de clientes empresariais.
Objectivo	Desenvolver um sistema que permita controlar o inventário de combustíveis por posto e por tanque, registar abastecimentos, gerir contratos com clientes empresariais e emitir relatórios de consumo e receita.
Funcionalidades	• Cadastro de postos de combustível com localização, número de bombas e responsável • Gestão de tanques por posto com tipo de combustível, capacidade e nível actual • Registo de abastecimentos com matrícula, tipo de combustível, quantidade e valor • Gestão de contratos com clientes empresariais com limites de crédito mensais • Controlo de reposição de combustível com registo de recepção de cisterna • Relatório de consumo por tipo de combustível, por posto e por período
Classes	Posto, Tanque, Bomba, Abastecimento, ClienteEmpresarial, Contrato, GestorRede
Relações	Posto agrega Tanque e Bomba; Abastecimento associa Bomba e Tanque; Contrato associa ClienteEmpresarial e Posto.
Conceitos POO	Composição Posto–Tanque–Bomba, colecções genéricas, enum (TipoCombustivel, EstadoTanque), interface (IAlertavel).
Excepções	TanqueInsuficienteException, LimiteCreditoExcedidoException, BombaIndisponivelException.
Testes	Cadastrar posto; registar abastecimento; verificar nível mínimo de tanque; processar contrato empresarial; emitir relatório de consumo.
Sugestões UML	Composição Posto–Tanque–Bomba; associação Abastecimento–Bomba; enum TipoCombustivel; interface IAlertavel.

Projecto 32: Sistema de Gestão de Empresa de Telecomunicações

Sistema de Gestão de Empresa de Telecomunicações	
Contexto	Uma operadora de telecomunicações necessita de um sistema para gerir os seus clientes, planos de serviço (voz, dados e pacotes), activaões, consumos e facturação mensal.

Objectivo	Implementar um sistema que suporte o ciclo de vida do cliente de uma operadora, desde a adesão e activação de plano até ao apuramento do consumo e emissão da factura mensal.
Funcionalidades	• Cadastro de clientes particulares e empresariais com dados de identificação • Gestão de planos de serviço com tarifários de voz, SMS e dados • Activação e desactivação de linhas telefónicas por cliente • Registo de consumos (chamadas, SMS, dados) com data, duração e custo • Cálculo da factura mensal com aplicação do tarifário e impostos • Controlo de pagamentos e gestão de contas em dívida
Classes	Cliente, ClienteParticular, ClienteEmpresarial, Linha, Plano, Consumo, Factura, GestorOperadora
Relações	ClienteParticular e ClienteEmpresarial herdam Cliente; Linha associa Cliente e Plano; Consumo compõe Linha; Factura agrega Consumo.
Conceitos POO	Herança abstracta (Cliente), polimorfismo (calcularDesconto), composição, enum (TipoConsumo, EstadoLinha), interface (IFacturavel).
Excepções	LinhaJaActivaException, PlanoIncompativelException, LimiteConsumoExcedidoException.
Testes	Cadastrar cliente; activar linha; registar consumo; calcular factura; desactivar linha; listar clientes em dívida.
Sugestões UML	Herança Cliente–ClienteParticular/Empresarial; composição Consumo–Linha; enum TipoConsumo; interface IFacturavel.

Projecto 33: Sistema de Gestão de Instituição de Microcrédito

Sistema de Gestão de Instituição de Microcrédito	
Contexto	Uma instituição de microcrédito presta serviços financeiros a pequenos empreendedores e necessita de um sistema para gerir candidaturas a empréstimos, análise de crédito, desembolsos, prestações e cobranças.
Objectivo	Desenvolver um sistema que suporte o ciclo completo do microcrédito, desde a candidatura até ao reembolso total do empréstimo, com controlo de prestações, juros e situação de incumprimento.

Funcionalidades	• Cadastro de mutuários com dados pessoais, actividade económica e historial de crédito • Submissão e análise de candidaturas a empréstimos com verificação de elegibilidade • Aprovação ou rejeição de candidaturas com registo de fundamentação • Geração do plano de amortização com cálculo de prestações e juros • Registo de pagamentos de prestações com actualização do saldo em dívida • Identificação e gestão de mutuários em incumprimento
Classes	Mutuario, Candidatura, Empréstimo, PlanoAmortizacao, Prestacao, Pagamento, GestorMicrocredito
Relações	Candidatura associa Mutuario; Empréstimo compõe PlanoAmortizacao; PlanoAmortizacao agrega Prestacao; Pagamento associa Prestacao.
Conceitos POO	Composição, colecções genéricas, enum (EstadoCandidatura, EstadoPrestacao), interface (IAnaliseavel), cálculo financeiro encapsulado.
Excepções	CandidaturaReprovadaException, PrestacaoJaPagaException, MutuarioEmIncumprimentoException.
Testes	Submeter candidatura; aprovar empréstimo; gerar plano; registar pagamento; identificar incumprimento; calcular saldo devedor.
Sugestões UML	Composição Empréstimo–PlanoAmortizacao–Prestacao; associação Pagamento–Prestacao; enum EstadoPrestacao; interface IAnaliseavel.

Projecto 34: Sistema de Gestão de Empresa de Distribuição de Água

Sistema de Gestão de Empresa de Distribuição de Água	
Contexto	Uma empresa pública de abastecimento de água necessita de um sistema para gerir os seus clientes, contadores de água, leituras mensais, cálculo de consumo e emissão de facturas.
Objectivo	Implementar um sistema que controle o ciclo de facturação da empresa distribuidora de água, desde o registo do contador até à emissão da factura com base nas leituras mensais efectuadas.
Funcionalidades	• Cadastro de clientes com categoria (residencial, comercial, industrial) e localização • Registo de contadores com número de série, data de instalação e estado

	• Registo de leituras mensais por contador com data e valor em metros cúbicos • Cálculo do consumo mensal por diferença entre leituras consecutivas • Aplicação de tarifários diferenciados por categoria de cliente e escalão de consumo • Emissão de facturas mensais e controlo de pagamentos
Classes	Cliente, Contador, Leitura, Consumo, Tarifario, Factura, GestorDistribuicao
Relações	Cliente tem Contador; Leitura compõe Contador; Consumo deriva de Leitura; Factura agrega Consumo e Tarifario.
Conceitos POO	Composição, colecções genéricas, enum (CategoriaCliente, EstadoContador), interface (ICalculavelConsumo).
Excepções	LeituraInvalidaException, ContadorNaoRegistadoException, ConsumoNegativoException.
Testes	Cadastrar cliente e contador; registar leitura; calcular consumo; aplicar tarifário; emitir factura; registar pagamento.
Sugestões UML	Composição Leitura–Contador; composição Factura–Consumo; enum CategoriaCliente; interface ICalculavelConsumo.

9. Critérios Obrigatórios de Implementação

Todos os projectos devem observar rigorosamente os critérios de implementação descritos nesta secção. O incumprimento de qualquer um dos critérios marcados como obrigatórios implicará dedução directa na pontuação atribuída.

9.1. Encapsulamento

Todos os atributos das classes devem ser declarados como privados (`private`) ou protegidos (`protected`), sendo o acesso externo efectuado exclusivamente através de propriedades (`properties`) em C#, com os modificadores `get` e `set` adequados. Não são admitidos campos públicos sem justificação documentada.

9.2. Herança

Sempre que o domínio do problema justifique uma relação de especialização/generalização, deve ser implementada herança entre classes. A classe base deve ser, preferencialmente, abstracta quando não existe instanciação directa.

9.3. Polimorfismo

Devem ser identificadas situações de polimorfismo de substituição (`override` de métodos virtuais ou abstractos) e polimorfismo de sobrecarga (`overload`). A utilização de referências polimórficas através de tipos base ou interfaces é obrigatória onde aplicável.

9.4. Classes Abstractas e Interfaces

Cada projecto deve incluir pelo menos uma classe abstracta ou uma interface. As interfaces devem ser definidas com nomes iniciados por 'I' (convenção C#) e devem representar contratos comportamentais relevantes para o domínio.

9.5. Composição e Agregação

As relações de composição (ciclo de vida dependente) e agregação (ciclo de vida independente) devem ser correctamente modeladas e implementadas. A composição deve ser reflectida na criação e destruição controlada dos objectos constituintes.

9.6. Colecções Genéricas

A gestão de colecções de objectos deve ser efectuada exclusivamente com colecções genéricas do namespace `System.Collections.Generic`: `List<T>`, `Dictionary<TKey, TValue>`, `Queue<T>`, `Stack<T>`, `LinkedList<T>` ou similares, conforme a necessidade do domínio.

9.7. Tratamento de Excepções

Devem ser definidas e implementadas pelo menos três (3) excepções personalizadas, derivadas da classe *Exception* ou de uma das suas subclasses *standard*. As excepções devem ser lançadas com *throw* em situações de erro de negócio e capturadas com blocos *try-catch-finally* nos pontos adequados.

9.8. Validação de Dados

Todos os dados introduzidos pelo utilizador devem ser validados antes de serem processados. A lógica de validação deve estar encapsulada nas classes de domínio ou em classes utilitárias dedicadas, nunca dispersa no *Program.cs*.

9.9. Separação de Responsabilidades

O código deve ser organizado de forma a que cada classe tenha uma responsabilidade claramente definida (Princípio da Responsabilidade Única). O *Program.cs* deve ser exclusivamente responsável pela orquestração do menu e pela interacção com o utilizador, delegando toda a lógica de negócio às classes adequadas.

9.10. Organização em Pastas e Namespaces

O projecto deve ser obrigatoriamente organizado em pastas correspondentes a namespaces distintos, conforme descrito na secção 10 desta proposta.

10. Estrutura Recomendada para Cada Projecto em C#

A estrutura seguinte representa a organização recomendada para todos os projectos. A adopção desta estrutura é obrigatória e será avaliada como parte dos critérios de organização do código.

Pasta / Ficheiro	Conteúdo e Responsabilidade
Models/	Classes de domínio: entidades, enumerações e classes de valor. Ex: <i>Produto.cs</i> , <i>TipoProduto.cs</i>
Interfaces/	Interfaces do sistema. Ex: <i>ICalculavel.cs</i> , <i>INotificavel.cs</i> , <i>IRepositorio.cs</i>
Services/	Classes de serviço com lógica de negócio. Ex: <i>GestorInventario.cs</i> , <i>ServicoRelatorio.cs</i>

Repositories/	(Quando aplicável) Repositórios de dados em memória. Ex: ProdutoRepository.cs
Exceptions/	Excepções personalizadas. Ex: StockInsuficienteException.cs, ProdutoNaoEncontrado.cs
Utils/	Classes utilitárias: validação, formatação, constantes. Ex: Validador.cs, Formatador.cs
Program.cs	Ponto de entrada. Instanciação do sistema e menu de navegação em consola.

Cada ficheiro .cs deve declarar o seu namespace correspondente à pasta em que se encontra. Por exemplo, uma classe Produto localizada na pasta `Models` deve declarar `namespace NomeProjecto.Models`. O `Program.cs` deve referenciar todos os namespaces necessários através de directivas `using`.

11. Orientações para a Elaboração do Diagrama de Classes UML

O diagrama de classes UML é um elemento avaliativo de elevado peso e deve reflectir fielmente a arquitectura do sistema implementado. As orientações seguintes são de cumprimento obrigatório.

11.1. Representação de Classes

Cada classe deve ser representada por um rectângulo dividido em três compartimentos: nome da classe (compartimento superior), atributos (compartimento intermédio) e métodos (compartimento inferior). As classes abstractas devm ter o nome em itálico ou com o estereótipo `<<abstract>>`.

11.2. Atributos e Métodos

Os atributos devem indicar o modificador de acesso (+ para *public*, - para *private*, # para *protected*), o nome e o tipo de dados. Os métodos devem indicar o modificador de acesso, o nome, os parâmetros com tipos e o tipo de retorno.

11.3. Relações entre Classes

- Herança (generalização): seta de linha sólida com ponta triangular vazia, da classe derivada para a classe base.

- Realização de interface: seta de linha tracejada com ponta triangular vazia, da classe implementadora para a interface.
- Associação: linha sólida simples entre classes, com multiplicidades em ambas as extremidades.
- Agregação: linha sólida com losango vazio na extremidade da classe agregadora.
- Composição: linha sólida com losango preenchido na extremidade da classe composta.
- Dependência: linha tracejada com seta aberta.

11.4. Multiplicidades

As multiplicidades devem ser indicadas em todas as associações, agregações e composições. Notação: 1 (exactamente um), 0..1 (zero ou um), 0..* ou * (zero ou mais), 1..* (um ou mais), n..m (entre n e m).

11.5. Interfaces

As interfaces devem ser representadas com o estereótipo <<interface>> acima do nome. Os métodos declarados na interface devem ser listados no compartimento de métodos da classe-interface.

12. Requisitos de Documentação

Cada grupo deve entregar um relatório técnico em formato digital (DOCX), organizado segundo a estrutura descrita a seguir. A ausência ou insuficiência de qualquer secção implicará redução proporcional na pontuação atribuída à documentação.

12.1. Estrutura do Relatório Técnico

7. Página de rosto: título do projecto, nomes e números dos estudantes, unidade curricular, curso, instituição, ano lectivo e data.
8. Índice automático com numeração de páginas.
9. Introdução: descrição sucinta do problema abordado e dos objectivos do sistema.
10. Análise de Requisitos: listagem dos requisitos funcionais e não funcionais, casos de utilização identificados.
11. Modelo UML: diagrama de classes completo com descrição textual das relações representadas.

12. Descrição das Principais Classes: explicação do papel de cada classe no sistema, atributos relevantes e métodos de maior complexidade.
13. Descrição dos Algoritmos Utilizados: pseudocódigo ou descrição textual dos algoritmos de maior relevância (cálculos, ordenações, pesquisas).
14. Instruções de Compilação e Execução: procedimento passo-a-passo para compilar e executar o projecto.
15. Evidências de Teste: descrição dos casos de teste executados, resultados esperados vs. resultados obtidos, capturas de ecrã da consola.
16. Conclusão: reflexão crítica sobre os resultados obtidos, dificuldades encontradas e competências desenvolvidas.
17. Referências: lista de fontes consultadas em formato APA ou IEEE. (Usar um gestor bibliográfico)

13. Critérios de Avaliação com Pesos Percentuais

Critério de Avaliação	Descrição	Peso
Qualidade da Análise do Problema	Identificação correcta de requisitos, completude da análise funcional e não funcional	10%
Correcção do Diagrama UML	Exactidão, completude e conformidade com a notação UML standard	15%
Aplicação dos Conceitos de POO	Uso correcto de encapsulamento, herança, polimorfismo, abstracção, composição e interfaces	20%
Organização e Legibilidade do Código	Convenções de nomenclatura, indentação, separação por pastas e namespaces	10%
Funcionamento do Sistema	Correcta execução de todas as funcionalidades previstas sem erros críticos	15%
Tratamento de Erros e Validações	Cobertura de excepções personalizadas e validação robusta de entradas	10%
Testes Realizados	Abrangência, pertinência e documentação dos testes executados	5%
Documentação Técnica	Qualidade e completude do relatório técnico	5%
Apresentação e Defesa Oral	Clareza expositiva, domínio técnico e capacidade de resposta	7%
Participação Individual	Contribuição equitativa e domínio individual demonstrado	3%
TOTAL		100%

A classificação final é expressa na escala de 0 a 20 valores, correspondendo cada percentagem a uma fracção proporcional dessa escala. O arredondamento é efectuado às décimas.

14. Modelo de Grelha de Avaliação (0–20 Valores)

Critério	Pontuação Max.	Nota Obtida	Observações	Peso
1. Análise do Problema	0–2			10%
2. Diagrama UML	0–3			15%
3. Conceitos POO	0–4			20%
4. Organização do Código	0–2			10%
5. Funcionamento do Sistema	0–3			15%
6. Tratamento de Erros	0–2			10%
7. Testes	0–1			5%
8. Documentação	0–1			5%
9. Apresentação Oral	0–1,4			7%
10. Participação Individual	0–0,6			3%
CLASSIFICAÇÃO FINAL (0–20 valores)				

A grelha deve ser preenchida pelo docente avaliador para cada grupo, podendo ser complementada com uma grelha de participação individual aplicada durante a apresentação e defesa.

15. Orientações para a Apresentação Pública

15.1. Duração e Estrutura

Cada grupo dispõe de um período total de vinte a vinte e cinco (20–25) minutos para a apresentação, organizado do seguinte modo:

- Apresentação geral do projecto e contexto do problema (3–4 minutos)
- Explicação e análise do diagrama de classes UML (4–5 minutos)
- Demonstração funcional do sistema em execução na consola (5–7 minutos)
- Explicação de excertos de código seleccionados pelo docente ou pelo grupo (3–4 minutos)
- Sessão de questões técnicas pelo docente e defesa pelo grupo (3–5 minutos)

15.2. Demonstração do Sistema

A demonstração deve ser efectuada com o projecto compilado e executável na máquina utilizada na apresentação. Devem ser demonstrados, pelo menos, três cenários de utilização, incluindo um cenário de erro tratado por excepção personalizada.

15.3. Explicação do Modelo UML

O diagrama de classes deve ser projectado e explicado com indicação das relações entre classes, das multiplicidades adoptadas e das decisões de design que justificam a arquitectura proposta.

15.4. Demonstração do Código

Devem ser apresentados e explicados os excertos de código que ilustrem a aplicação dos conceitos de POO: uma classe abstracta ou interface, um exemplo de herança com override, o tratamento de uma excepção personalizada e uma operação sobre uma colecção genérica.

15.5. Defesa Técnica

O docente poderá colocar questões técnicas a qualquer membro do grupo individualmente. A capacidade de resposta individual é avaliada no critério de participação individual e contribui para a diferenciação intra-grupo.

16. Resultados Esperados da Avaliação

Ao concluir a avaliação prática, os estudantes deverão ser capazes de demonstrar as seguintes competências:

16.1. Competências de Modelação

- Analisar um problema do domínio real e identificar os seus requisitos funcionais e não funcionais com rigor e completude.
- Elaborar um diagrama de classes UML completo, correcto e conforme com a notação standard, reflectindo todas as entidades, relações e multiplicidades do sistema.

16.2. Competências de Implementação

- Implementar em C# um sistema orientado a objectos com aplicação correcta de encapsulamento, herança, polimorfismo, abstracção e composição.
- Definir e utilizar interfaces e classes abstractas como mecanismos de contrato comportamental e de extensibilidade.
- Gerir colecções de objectos com colecções genéricas e implementar operações de pesquisa, ordenação e filtragem sobre essas colecções.
- Implementar e lançar excepções personalizadas em situações de erro de negócio, garantindo o tratamento adequado em todos os pontos críticos do sistema.
- Organizar o código-fonte em pastas e namespaces com separação clara de responsabilidades.

16.3. Competências de Teste e Qualidade

- Definir e executar casos de teste relevantes que cubram os cenários principais e os casos de erro do sistema.
- Produzir documentação técnica de qualidade que descreva com rigor o sistema desenvolvido.

16.4. Competências Comunicacionais

- Apresentar e defender publicamente um produto de software de forma estruturada, clara e tecnicamente fundamentada.
- Responder com segurança a questões técnicas sobre as decisões de design e de implementação tomadas.